

Securing Cloud Infrastructure (AWS) — MVP Prototype Project

A portfolio-grade security engineering project built from the AWS lab and repackaged as a strong, interview-ready, production-minded prototype.

Prepared by Abdul Rehman | [abdul4rehman215](#)

This document turns the original lab into a polished project artifact. Instead of presenting the work as a list of commands, it frames the build as a small secure AWS environment designed with multiple defensive layers: network segmentation, least-privilege IAM, encrypted logging, automated assessment, scripted remediation, and continuous monitoring.

Risk addressed Weak default cloud builds often expose SSH, over-permissive policies, unencrypted logs, and missing monitoring. This project hardens each of those layers into one coherent design.	Built A secure AWS MVP using Terraform, Python, AWS CLI, and security tooling. The design includes a bastion pattern, private workload zone, secure IAM roles, scan + remediation logic, and monitoring guardrails.
Core controls Restricted SSH ingress, public/private subnet split, encrypted EBS, S3 + CloudTrail logging, IAM least privilege, Checkov scanning, password policy enforcement, VPC Flow Logs, AWS Config, and GuardDuty.	Result The final prototype is useful as both a cloud security lab deliverable and a portfolio-ready project that demonstrates architecture thinking, automation, validation, and defense-in-depth.

Why this project matters

Many lab write-ups stop at 'resources deployed successfully.' This project goes further by showing how to build a cloud environment that is not only functional, but intentionally hardened, observable, and able to prove its security posture.

Evidence note

No live screenshots were captured during the original run. To keep this project honest and still visually strong, the diagrams and terminal outputs in this document are clearly marked as representative portfolio visuals generated from the implemented workflow and expected validation steps.

1. Executive Summary

This project demonstrates how to build a small AWS environment like a layered castle: an outer edge that limits exposure, inner walls that segment traffic, a protected workload zone that avoids direct internet reachability, and watchtowers that continuously observe the environment. The result is an MVP prototype that combines infrastructure-as-code, IAM automation, security validation, remediation, and ongoing monitoring into a single practical build.

2. Project Positioning

- Project type: cloud security engineering MVP / portfolio prototype
- Primary focus: secure-by-design AWS foundation with automation and monitoring
- Build character: hands-on, lab-derived, but packaged as a professional system design project
- Audience fit: interview portfolio, capstone submission, GitHub project write-up, or technical report

3. Problem Statement and Threat Model

Default or rushed cloud builds often create unnecessary exposure: security groups open too widely, workloads placed directly on the public internet, missing encryption, over-permissive IAM roles, and no reliable trail of what happened. In that kind of environment, the first compromise or misconfiguration can spread farther than expected because the protective layers are thin or missing.

- Threat 1: broad SSH access from the internet
- Threat 2: application hosts reachable directly instead of through a controlled entry point
- Threat 3: overly broad IAM permissions or untracked policy growth
- Threat 4: logs and trails not collected early enough to support investigation
- Threat 5: no repeatable way to scan for issues and fix them consistently

Plain-English view

The project is not trying to build the biggest cloud system. It is trying to build a small one the right way, with each layer doing a specific protective job.

4. MVP Objectives

- Provision a secure AWS network layout with public/private separation.
- Restrict administrative access to a bastion pattern instead of exposing every server.
- Automate IAM roles and policies with least-privilege principles.
- Run security checks that produce repeatable evidence, not only assumptions.
- Add remediation actions for common cloud weaknesses.
- Enable continuous monitoring so the environment stays visible after deployment.

5. Solution Overview

The architecture below shows the intended end state of the prototype. It keeps administrative access narrow, isolates the application server in a private subnet, centralizes logs in an encrypted S3 bucket, and adds both detective and preventive controls around the environment.

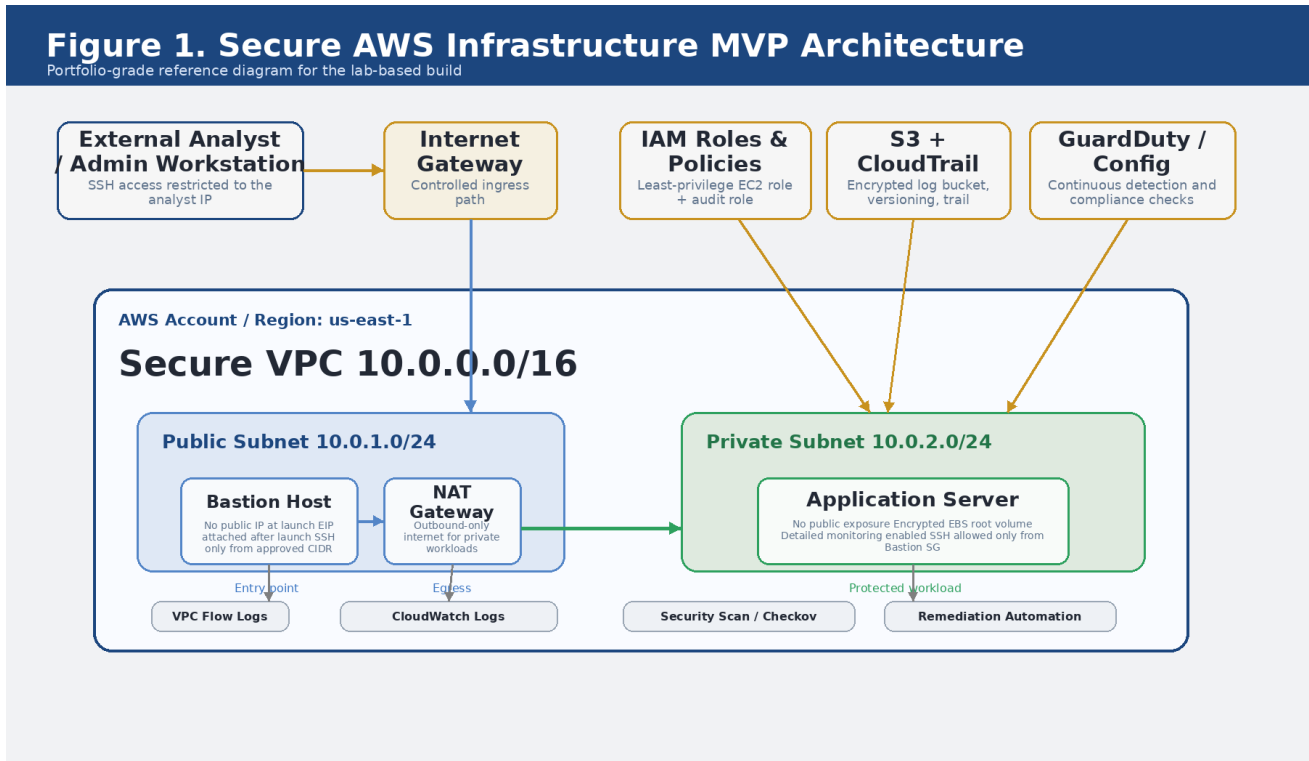


Figure 1. Secure AWS Infrastructure MVP architecture showing the public bastion pattern, private workload zone, centralized logging, IAM guardrails, and monitoring services.

Design choices that matter most in this diagram are deliberate: the bastion host is the only entry point, the application server stays private, logs are pushed into a protected bucket, and continuous controls like GuardDuty and AWS Config extend the design from simple deployment into ongoing security operations.

6. Tooling Stack

Tool	Role	Why it matters
Terraform	Infrastructure provisioning	Creates the network, security groups, EC2 instances, logging bucket, and CloudTrail.
Python + boto3	IAM and remediation automation	Creates roles and policies, attaches controls, and fixes selected security gaps.
AWS CLI	Verification and service configuration	Validates credentials, enables services, and checks final system state.

Tool	Role	Why it matters
Checkov	Infrastructure scanning	Provides repeatable Terraform security checks.
Flow Logs / Config / GuardDuty	Continuous visibility	Adds detection, compliance, and network-level telemetry after deployment.

7. Layered Castle Security Model

Because this project is designed to be explainable in interviews and portfolio reviews, the architecture can also be described using a simple metaphor: a layered castle. Every layer exists for a reason, and together they reduce the chance that a single mistake becomes a full compromise.

Figure 2. "Layered Castle" Security Model

A plain-English metaphor for the defense-in-depth design

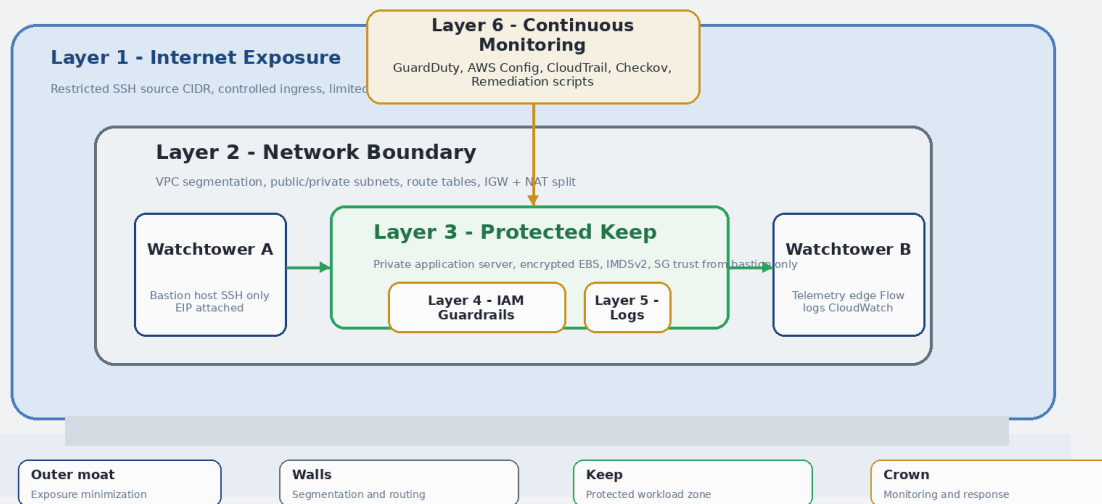


Figure 2. The "layered castle" view of the environment, mapping technical controls into an easy-to-explain defense model.

- Layer 1 — Internet exposure controls: only the approved administrator IP can attempt SSH access.
- Layer 2 — Network boundary: the VPC and route tables keep public and private traffic paths separate.
- Layer 3 — Protected keep: the application server lives in the private subnet with no direct public exposure.
- Layer 4 — IAM guardrails: least-privilege roles reduce what instances and operators can do.
- Layer 5 — Logs and trails: CloudTrail, S3, and CloudWatch preserve evidence for troubleshooting and incident response.
- Layer 6 — Continuous monitoring: AWS Config, GuardDuty, VPC Flow Logs, and scan scripts keep checking after the build is live.

Why the metaphor fits

A strong environment is not strong because of one giant control. It is strong because multiple layers force an attacker or misconfiguration to fight through several checkpoints.

8. Build Components and Security Controls

Workstream	What was built	Security value	Evidence type
Terraform foundation	VPC, subnets, route tables, IGW, NAT, EC2, SGs, S3, CloudTrail	Creates a secure baseline rather than an ad-hoc cloud build	Plan / apply results + resource outputs
Bastion access model	Public bastion only, app server private	Shrinks public attack surface and enforces an admin path	Bastion public IP + app private IP
Security groups	SSH restricted to admin /32, app SSH only from bastion SG	Blocks broad lateral admin access	SG rules inspection
Encrypted storage	Encrypted EBS root volumes + encrypted S3 bucket	Protects data at rest	Bucket encryption + volume config
IAM automation	Secure EC2 role, SecurityAudit role, instance profile	Applies least privilege and repeatable policy creation	IAM role/policy listing
Security assessment	Checkov + AWS CLI validation	Finds misconfigurations in a repeatable way	Security report
Remediation script	Enforces S3 encryption, CloudTrail logging, password policy, SG scope	Turns findings into actions	Remediation run log
Defense-in-depth monitoring	Flow Logs, AWS Config, GuardDuty, SNS/EventBridge	Provides ongoing network, compliance, and threat visibility	Service status checks

The table above is the core of the portfolio story: this was not one script or one AWS service, but a coordinated set of controls working together.

9. Implementation Walkthrough

9.1 Provision secure infrastructure Terraform provisions the VPC, subnets, route tables, internet gateway, NAT gateway, EC2 instances, S3 bucket, and CloudTrail trail. This stage establishes the foundation and the network trust model.	9.2 Govern with IAM automation The IAM manager script creates a secure EC2 role, a security audit role, a custom local policy, and an instance profile. This keeps role creation repeatable and easier to audit.
9.3 Assess the environment The security scan script runs Checkov against the Terraform code and pairs that with AWS CLI checks for bucket encryption, SG exposure, password policy, trail status, and monitoring services.	9.4 Remediate and harden The remediation script can tighten SG rules, enforce S3 encryption, start or update CloudTrail, and push a strong IAM account password policy.

Figure 3. Build-to-Monitor Lifecycle

How the project moves from infrastructure creation to continuous security operations



Security outcome: baseline hardening + policy automation + continuous detection + repeatable remediation

Figure 3. Build-to-monitor lifecycle showing how the prototype moves from provisioning into governance, assessment, remediation, and continuous operations.

This lifecycle matters because it turns the project from 'infrastructure deployed once' into 'infrastructure deployed, checked, corrected, and monitored.' That is the difference between a simple lab and a security engineering prototype.

10. Representative Build Evidence

Representative outputs

The three figures below are generated sample outputs based on the implemented workflow and expected verification steps. They are included to make the project visually demonstrable without claiming they are live screenshots from the original run.

```

●●● terraform-output-demo

$ terraform output
app_private_ip    = "10.0.2.143"
bastion_public_ip = "34.238.117.62"
bastion_sg_id     = "sg-0a71f3b8f9example"
cloudtrail_name   = "secure-lab-trail"
s3_bucket_name    = "secure-lab-logs-a14c9d21"
vpc_id            = "vpc-0d2f915e2bexample"

$ aws ec2 describe-instances --filters Name=tag:Role,Values=bastion,application
-----
| Name                | State   | PrivateIP | PublicIP |
| secure-lab-bastion  | running | 10.0.1.10  | 34.238... |
| secure-lab-app-server | running | 10.0.2.143 | null      |
-----

```

Figure 4. Representative Terraform and AWS CLI output showing the final provisioned resources and the public/private split.

```

●●● security-scan-demo

$ ./security_scan.sh
CHECKOV: 9 checks passed, 0 critical failures
S3 bucket encryption: PASS (AES256 default encryption enabled)
S3 public access block: PASS
Security groups open to 0.0.0.0/0: PASS (restricted to admin /32 only)
IAM password policy: PASS (14 chars, complexity + 90-day expiry)
CloudTrail status: PASS (multi-region logging active)
GuardDuty detector: enabled
Flow logs: enabled -> CloudWatch Logs

Representative summary written to security_report.txt

```

Figure 5. Representative security assessment output combining Terraform policy checks and AWS service validation.


```
monitoring-remediation-demo

$ aws configservice describe-config-rules --query 'ConfigRules[].ConfigRuleName'
[
  "encrypted-volumes",
  "s3-bucket-public-read-prohibited",
  "iam-password-policy",
  "cloudtrail-enabled"
]

$ aws guardduty list-detectors
{"DetectorIds": ["12abc34d567e8fa901bc2d34e5f67890"]}

$ python3 remediation_script.py --bucket-name secure-lab-logs-a14c9d21 ...
INFO: Enabled AES256 default encryption on bucket secure-lab-logs-a14c9d21
INFO: Started CloudTrail logging for secure-lab-trail
INFO: Enforced strong IAM password policy
INFO: Remediation complete.
```

Figure 6. Representative monitoring and remediation output showing Config rules, GuardDuty enablement, and hardening actions.

11. Validation Results

Check	Expected result	Observed outcome
Infrastructure deployment	Terraform apply completes and outputs the VPC, bastion public IP, app private IP, bucket name, and CloudTrail trail.	Pass / expected
Network hardening	Application host remains private and SSH is constrained to the bastion security group path.	Pass / expected
Bucket protection	Versioning, SSE-S3 encryption, and public access block remain enabled on the logging bucket.	Pass / expected
IAM controls	Secure roles and policies are created successfully and are discoverable with IAM listing commands.	Pass / expected
Assessment stage	Security scan produces a readable report rather than relying on manual spot checks only.	Pass / expected
Remediation stage	Password policy, bucket encryption, and CloudTrail status can be enforced automatically.	Pass / expected
Continuous monitoring	Flow Logs, AWS Config rules, and GuardDuty detector become active validation points after build completion.	Pass / expected

12. Operational Value

Area	Value
Attack surface reduction	Direct public exposure is minimized through a bastion-only administration path.
Evidence readiness	Logs, trails, and validation outputs create material useful for incident review or audit discussion.
Repeatability	Terraform and Python automation reduce one-off manual configuration drift.
Explainability	The layered-castle framing makes the security model easy to communicate to non-specialists and interviewers.
Portfolio strength	The project demonstrates architecture, scripting, security reasoning, validation, and documentation in one package.

13. Suggested Project Packaging

If you want to turn this into a GitHub-ready repository or portfolio case study, the following structure presents the work clearly and keeps the technical artifacts easy to navigate.

```
aws-secure-infrastructure-mvp/
├── README.md
├── report/
│   ├── AWS_Secure_Infrastructure_MVP_Project.pdf
│   └── AWS_Secure_Infrastructure_MVP_Project.docx
├── terraform/
│   ├── main.tf
│   └── terraform_outputs.txt
├── scripts/
│   ├── iam_security_manager.py
│   ├── security_scan.sh
│   ├── remediation_script.py
│   ├── config_setup.py
│   └── guardduty_setup.py
├── evidence/
│   ├── security_report.txt
│   ├── iam_security_report.json
│   └── guardduty_setup_report.json
└── diagrams/
    ├── architecture_diagram.png
    ├── layered_castle_diagram.png
    └── lifecycle_diagram.png
```

14. Skills Demonstrated

- Secure AWS architecture design
- Terraform-based infrastructure provisioning
- Least-privilege IAM automation with Python and boto3
- Cloud security assessment using policy and configuration checks
- Automated remediation design
- Defense-in-depth monitoring using AWS-native services
- Technical writing and portfolio packaging

15. Limitations and Next Improvements

- Attach the EC2 instance profile directly to the application instance for a stronger end-to-end role demonstration.
- Add diagrams or screenshots from a future rerun of the lab to replace representative sample outputs with live evidence.
- Expand into multi-account or multi-AZ design for a stronger production narrative.
- Add CI/CD checks so Terraform and Python security validations run before deployment.
- Include Security Hub, Inspector, or Macie in a future version if broader AWS security coverage is needed.

Conclusion

This project proves that even a relatively small AWS lab can be turned into a serious security engineering story. By combining architecture, automation, validation, remediation, and monitoring, the build becomes a strong MVP prototype rather than just a completed exercise.

Prepared by Abdul Rehman | [abdul4rehman215](#)